

Manuale Teorico e Implementativo Completo: Algoritmi del `general_solver.py` in Gasflash

Gasflash Development Team

28 aprile 2026

Sommario

Questo documento costituisce la referenza tecnica definitiva ed esaustiva per il modulo `general_solver.py` utilizzato all'interno dell'ambiente Gasflash. L'obiettivo di questo testo è fornire una disamina chirurgica di ogni singola riga di logica matematica, fisica e computazionale presente nell'algoritmo. Verranno trattati: la derivazione delle Equazioni di Eulero Quasi-1D, la discretizzazione ai Volumi Finiti, l'analisi spettrale per il Metodo di Riemann Approssimato di Roe, l'applicazione del Correttore di Harten, l'implementazione del trattamento dei termini sorgente (geometria, Fanno, Rayleigh), la modellazione rigorosa delle condizioni al contorno caratteristiche, e le tecniche avanzate di programmazione concorrente e compilazione JIT tramite Numba adottate per massimizzare le prestazioni. Questo manuale è stato redatto affinché un neofita, partendo dai principi primi, possa arrivare a comprendere e riprodurre l'intero solutore CFD.

Indice

1 Introduzione alla Fluidodinamica Computazionale (CFD) Unidimensionale

1.1 Scopo del Solutore

La simulazione dei flussi comprimibili all'interno di condotti, ugelli e tubazioni è un problema classico dell'ingegneria aerospaziale e meccanica. Quando le variazioni della sezione trasversale del condotto avvengono in modo graduale, o quando la lunghezza del condotto è significativamente maggiore del suo diametro, il flusso tridimensionale reale può essere approssimato in maniera eccellente assumendo che tutte le proprietà del fluido (pressione, velocità, densità) varino esclusivamente lungo l'asse longitudinale del condotto (coordinata x). Questo approccio prende il nome di **Approssimazione Quasi-1D**.

1.2 Gas Ideale e Termodinamica

L'algoritmo assume che il fluido in esame sia un Gas Perfetto (o Ideale). Le proprietà del gas sono governate dall'equazione di stato:

$$p = \rho RT \quad (1)$$

dove p è la pressione statica, ρ la densità, R la costante specifica del gas e T la temperatura statica. L'energia interna specifica e e l'entalpia specifica h sono definite come:

$$e = c_v T = \frac{R}{\gamma - 1} T = \frac{p}{(\gamma - 1)\rho} \quad (2)$$

$$h = e + \frac{p}{\rho} = c_p T = \frac{\gamma p}{(\gamma - 1)\rho} \quad (3)$$

dove $\gamma = c_p/c_v$ è il calore specifico rapportato (costante adiabatica).

2 Modello Matematico: Equazioni di Eulero Quasi-1D

2.1 Forma Conservativa

Le equazioni fondamentali che governano la dinamica del gas sono la conservazione della massa, della quantità di moto e dell'energia. In assenza di viscosità interna del gas e di conduzione termica, ma in presenza di variazioni di sezione $A(x)$, attrito alle pareti e scambio termico con l'esterno, le Equazioni di Navier-Stokes si riducono al sistema di Eulero Quasi-1D. Per garantire la corretta cattura delle discontinuità (Onde d'Urto), il sistema deve essere scritto rigorosamente in **forma conservativa forte**:

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial (\mathbf{F} \cdot A)}{\partial x} = \mathbf{S} \quad (4)$$

I vettori di stato sono definiti come segue:

Vettore delle Variabili Conservative $\mathbf{U}$:

$$\mathbf{U} = \begin{pmatrix} \rho A \\ \rho u A \\ E_t A \end{pmatrix} = \begin{pmatrix} U_0 \\ U_1 \\ U_2 \end{pmatrix} \quad (5)$$

Dove E_t è l'energia totale per unità di volume:

$$E_t = \frac{p}{\gamma - 1} + \frac{1}{2} \rho u^2 \quad (6)$$

Vettore dei Flussi $\mathbf{F}$:

$$\mathbf{F} = \begin{pmatrix} \rho u \\ \rho u^2 + p \\ u(E_t + p) \end{pmatrix} = \begin{pmatrix} \rho u \\ \rho u^2 + p \\ \rho u H \end{pmatrix} \quad (7)$$

Dove H è l'entalpia totale specifica:

$$H = \frac{E_t + p}{\rho} = \frac{\gamma p}{(\gamma - 1)\rho} + \frac{1}{2}u^2 \quad (8)$$

Vettore dei Termini Sorgente $\mathbf{S}$:

$$\mathbf{S} = \begin{pmatrix} 0 \\ p \frac{dA}{dx} - F_\tau \\ \dot{q}A \end{pmatrix} \quad (9)$$

Analizziamo i termini sorgente:

- $p \frac{dA}{dx}$: rappresenta la componente assiale della spinta di pressione esercitata dalle pareti del condotto inclinate contro il fluido.
- F_τ : rappresenta lo stress di taglio (attrito) alla parete per unità di lunghezza. Viene valutato mediante il coefficiente di Fanning $f_{Fanning}$ come: $F_\tau = 2f_{Fanning}\rho|u|\frac{A}{D}$, dove D è il diametro idraulico. Il valore assoluto assicura che l'attrito si opponga sempre al moto.
- $\dot{q}A$: rappresenta il flusso di calore per unità di lunghezza aggiunto al fluido (Watt al metro), comunemente denominato flusso di Rayleigh.

3 Discretizzazione Spaziale: Il Metodo ai Volumi Finiti

3.1 Definizione della Griglia

Il dominio computazionale di lunghezza L è diviso in N_x celle (o volumi di controllo) adiacenti. Ogni cella i ha una dimensione $\Delta x = L/N_x$. I centri delle celle si trovano in x_i , mentre le interfacce tra le celle si trovano in $x_{i-1/2}$ e $x_{i+1/2}$. Nel codice `general_solver.py`, le celle sono indicizzate da 0 a $N_x - 1$. I confini (interfacce) sono indicizzati da 0 a N_x .

3.2 Integrazione Conservativa

Integrando il sistema di equazioni su un volume di controllo Δx tra i tempi t^n e t^{n+1} :

$$\int_{t^n}^{t^{n+1}} \int_{x_{i-1/2}}^{x_{i+1/2}} \left(\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial (\mathbf{F} \cdot A)}{\partial x} \right) dx dt = \int_{t^n}^{t^{n+1}} \int_{x_{i-1/2}}^{x_{i+1/2}} \mathbf{S} dx dt \quad (10)$$

Assumendo che $\mathbf{U}_i$ rappresenti la media spaziale delle variabili all'interno della cella i , otteniamo la formulazione esplicita per l'avanzamento nel tempo (Eulero in avanti):

$$\mathbf{U}_i^{n+1} = \mathbf{U}_i^n - \frac{\Delta t}{\Delta x} \left[\mathbf{F}_{i+1/2}^* A_{i+1/2} - \mathbf{F}_{i-1/2}^* A_{i-1/2} \right] + \Delta t \mathbf{S}_i^n \quad (11)$$

L'aspetto più critico di qualsiasi solutore CFD è la determinazione del **flusso numerico all'interfaccia** $\mathbf{F}_{i+1/2}^*$. Utilizzare una semplice media aritmetica degli stati adiacenti genera instabilità esplosive per via della violazione del dominio di dipendenza fisico delle onde. La soluzione a questo problema richiede la soluzione del "Problema di Riemann".

4 Il Risolutore di Riemann Approssimato di Roe

4.1 Il Problema di Riemann

All'interfaccia $x_{i+1/2}$, abbiamo due stati costanti: lo stato di sinistra $\mathbf{U}_L$ (proveniente dalla cella i) e lo stato di destra $\mathbf{U}_R$ (proveniente dalla cella $i+1$). Questa discontinuità iniziale si evolve generando tre onde fondamentali: un urto (o espansione), una discontinuità di contatto e una seconda espansione (o urto).

Phil Roe (1981) propose un metodo fenomenale: invece di risolvere esattamente il problema non lineare (che richiederebbe iterazioni costose all'interno di ogni singola cella per ogni istante di tempo), possiamo linearizzare localmente il problema introducendo una matrice Jacobiana costante speciale $\tilde{\mathbf{J}}(\mathbf{U}_L, \mathbf{U}_R)$.

4.2 Medie di Roe

Il cuore del metodo consiste nel trovare una media (detta *Roe Average*) tra lo stato L e lo stato R in modo tale che la Jacobiana calcolata con questi stati medi soddisfi rigorosamente la proprietà di conservazione:

$$\mathbf{F}(\mathbf{U}_R) - \mathbf{F}(\mathbf{U}_L) = \tilde{\mathbf{J}} \cdot (\mathbf{U}_R - \mathbf{U}_L) \quad (12)$$

Per il gas perfetto, le medie di Roe per densità, velocità ed entalpia totale sono calcolate mediante una pesatura legata alla radice quadrata della densità:

$$\tilde{\rho} = \sqrt{\rho_L \rho_R} \quad (13)$$

$$\tilde{u} = \frac{\sqrt{\rho_L} u_L + \sqrt{\rho_R} u_R}{\sqrt{\rho_L} + \sqrt{\rho_R}} \quad (14)$$

$$\tilde{H} = \frac{\sqrt{\rho_L} H_L + \sqrt{\rho_R} H_R}{\sqrt{\rho_L} + \sqrt{\rho_R}} \quad (15)$$

A partire da queste medie di Roe, la *Velocità del Suono di Roe* $\tilde{a}$ si ricava invertendo l'equazione dell'entalpia totale:

$$\tilde{a} = \sqrt{\max\left((\gamma - 1) \left(\tilde{H} - \frac{1}{2}\tilde{u}^2\right), 10^{-12}\right)} \quad (16)$$

La protezione al valore minimo 10^{-12} previene errori computazionali (radici di numeri negativi) durante transitori severi ed è visibile alla riga 16 di `general_solver.py`.

4.3 Struttura Spettrale (Autovalori e Autovettori)

La linearizzazione ci permette di scomporre la discontinuità iniziale in una somma di onde caratteristiche semplici, ognuna viaggiante ad una velocità dettata dagli **Autovalori** λ_k della Jacobiana di Roe, e portante una certa quantità di informazione fisica determinata dagli **Autovettori Destri** $\mathbf{K}_k$.

Autovalori (Velocità delle onde):

$$\lambda_1 = \tilde{u} \quad (\text{Onda entropica/discontinuità di contatto, viaggia col gas}) \quad (17)$$

$$\lambda_2 = \tilde{u} + \tilde{a} \quad (\text{Onda acustica destra, viaggia in avanti a monte}) \quad (18)$$

$$\lambda_3 = \tilde{u} - \tilde{a} \quad (\text{Onda acustica sinistra, viaggia indietro rispetto al gas}) \quad (19)$$

Autovettori Destri (Distribuzione fisica dell'onda):

$$\mathbf{K}_1 = \begin{pmatrix} 1 \\ \tilde{u} \\ \frac{1}{2}\tilde{u}^2 \end{pmatrix}, \quad \mathbf{K}_2 = \begin{pmatrix} 1 \\ \tilde{u} + \tilde{a} \\ \tilde{H} + \tilde{u}\tilde{a} \end{pmatrix}, \quad \mathbf{K}_3 = \begin{pmatrix} 1 \\ \tilde{u} - \tilde{a} \\ \tilde{H} - \tilde{u}\tilde{a} \end{pmatrix} \quad (20)$$

Forza delle Onde (Wave Strengths α_k): Queste grandezze determinano "quanto" grande è il salto associato a ciascuna onda. Proiettando la differenza degli stati originari $\Delta \mathbf{U} = \mathbf{U}_R - \mathbf{U}_L$ sulla base degli autovettori, otteniamo il salto in variabili primitive:

$$\alpha_1 = \Delta \rho - \frac{\Delta p}{\tilde{a}^2} \quad (21)$$

$$\alpha_2 = \frac{\Delta p + \tilde{\rho} \tilde{a} \Delta u}{2\tilde{a}^2} \quad (22)$$

$$\alpha_3 = \frac{\Delta p - \tilde{\rho} \tilde{a} \Delta u}{2\tilde{a}^2} \quad (23)$$

Nel codice (riga 32-36), $\Delta \rho = \rho_R - \rho_L$, $\Delta u = u_R - u_L$, $\Delta p = p_R - p_L$ e $\tilde{\rho} \tilde{a}$ corrisponde al termine di impedenza acustica.

4.4 Dissipazione Upwind e Calcolo del Flusso Finale

Il metodo di Roe appartiene alla famiglia degli schemi *Upwind* Centrali. Il flusso all'interfaccia viene calcolato partendo da una media aritmetica banale (instabile), per poi **sottrarre matematicamente** l'esatta quantità di "dissipazione artificiale", mirata specificatamente ad abbattere gli errori legati alla direzione delle onde.

$$\mathbf{F}_{i+1/2}^* = \frac{1}{2} [\mathbf{F}(\mathbf{U}_L) + \mathbf{F}(\mathbf{U}_R)] - \frac{1}{2} \sum_{k=1}^3 |\lambda_k| \alpha_k \mathbf{K}_k \quad (24)$$

Il secondo termine dell'equazione è comunemente chiamato Vettore Dissipazione (rappresentato dalla variabile `diss` nel codice). Come si evince, l'autovalore viene preso in modulo $|\lambda_k|$ perché le onde smorzano energia indipendentemente dalla loro direzione.

4.5 Il Correttore di Entropia di Harten (Entropy Fix)

Esiste un singolo difetto fatale nel solutore di Roe puro: non impone il Secondo Principio della Termodinamica (incremento dell'entropia). Di conseguenza, in punti transonici (dove il flusso accelera da subsonico a supersonico, $u \approx a$), un autovalore tende a zero ($\lambda \approx 0$). Questo annulla la dissipazione numerica associata, creando un'onda di discontinuità anti-fisica chiamata "Expansion Shock" (Urto di espansione).

Per evitare questo disastro numerico, `general_solver.py` implementa il Correttore di Harten. Se il modulo di una velocità d'onda $|\lambda_k|$ scende al di sotto di una piccola soglia δ , il suo modulo nel calcolo della dissipazione viene artificialmente ingrandito attraverso una parabola di raccordo raccordata senza spigoli (continuamente derivabile):

$$|\lambda_k|^* = \begin{cases} |\lambda_k| & \text{se } |\lambda_k| \geq \delta \\ \frac{\lambda_k^2 + \delta^2}{2\delta} & \text{se } |\lambda_k| < \delta \end{cases} \quad (25)$$

Nel codice Gasflash, il parametro empirico δ è impostato proporzionale alla velocità del suono mediata per scalare correttamente con la fisica del problema: $\delta = 0.4\tilde{a}$ (riga 22).

5 Integrazione Numerica e Stabilità

5.1 CFL e Condizione di Courant-Friedrichs-Lewy

Essendo il metodo di avanzamento nel tempo esplicito, la stabilità dell'algoritmo non è incondizionata. L'informazione (le onde) in un volume finito non deve viaggiare oltre i confini della cella in un singolo istante di tempo Δt , altrimenti la cella adiacente "saltirebbe" l'aggiornamento.

La velocità d'onda massima presente nell'intero dominio in un dato istante temporale è:

$$W_{S,\max} = \max_i (|u_i| + a_i) \quad (26)$$

Il timestep Δt calcolato dinamicamente è vincolato dal Numero di Courant (CFL):

$$\Delta t = \text{CFL} \cdot \frac{\Delta x}{W_{S,\max}} \quad (27)$$

Nel nostro solutore (riga 92), $\text{CFL} = 0.4$, un valore ampiamente conservativo e appropriato per schemi del primo ordine e volumi finiti quasi-1D (la stabilità lineare limite per schemi espliciti di Roe si attesta intorno a 1.0, ma i termini sorgente e non-lineari consigliano valori inferiori a 0.5 per garantire monotonia assoluta).

5.2 Gestione dei Limiti Fisici (Bounds Protection)

Soprattutto nella fase di "spin-up" o durante la formazione violenta di onde d'urto ad elevato rapporto di compressione, i residui numerici possono temporaneamente far convergere lo stato conservativo U_0 (massa totale) a valori nulli o persino leggermente negativi per approssimazione in virgola mobile.

Per garantire la stabilità computazionale incrollabile: 1. Durante l'estrazione delle variabili primitive dalle conservative, viene posta una condizione `if rho[i] < 1e-6: rho[i] = 1e-6`. 2. **Crucialmente**, questo troncamento viene riflesso immediatamente a ritroso nelle variabili conservative `U_curr[0, i] = rho[i] * A[i]`. Questo assicura che il sistema non stia solo "mascherando" il problema per il timestep in corso, ma corregga lo stato di conservazione reale. 3. Lo stesso vincolo logico è applicato all'energia e di riflesso alla pressione statica ($p > 10^{-5}$). Questo impedisce in modo sistematico l'insorgenza di divisioni per zero o estrazioni di radici immaginarie $\sqrt{p/\rho}$.

6 Condizioni al Contorno: La Fisica delle Boundary Conditions

Le condizioni al contorno (BC) in fluidodinamica comprimibile richiedono grande delicatezza perché il numero di variabili da imporre dipende da quante caratteristiche (autovalori) entrano nel dominio computazionale e quante ne escono.

6.1 BC di Ingresso (Stagnation Inlet)

Nell'implementazione pratica della "Pipe" gasdinamica, consideriamo che l'ingresso sia connesso a un serbatoio infinito. Sono noti due parametri fondamentali: - Pressione di Ristagno o Totale $P_{0,\text{in}}$ - Temperatura di Ristagno o Totale $T_{0,\text{in}}$

Poiché non conosciamo a priori la velocità con cui il gas deciderà di scorrere all'interno del tubo (dipende dalla pressione a valle), estrapoliamo la velocità in ingresso prendendola uguale alla primissima cella interna: $u_{\text{in}} = u_0$ (con protezione minima anti-ristagno zero 10^{-6}).

Poiché l'ingresso è idealizzato come processo Isentropico e Adiabatico dal serbatoio all'imboccatura del tubo, l'Entalpia Totale si conserva rigorosamente. Grazie a questo vincolo, possiamo derivare con certezza assoluta la temperatura statica all'inlet:

$$T_{\text{in}} = T_{0,\text{in}} - \frac{u_{\text{in}}^2}{2C_p} \quad (28)$$

Dove $C_p = \frac{\gamma R}{\gamma - 1}$. Trovata la temperatura statica T_{in} , tramite il rapporto di temperatura $\tau_{\text{in}} = T_{0,\text{in}}/T_{\text{in}}$, troviamo immediatamente densità e pressione statica da imporre nel vettore fantasma

(Ghost Cell) dell'ingresso sfruttando le relazioni isentropiche:

$$\rho_{\text{in}} = \rho_{0,\text{in}} (\tau_{\text{in}})^{-\frac{1}{\gamma-1}} \quad \text{essendo } \rho_{0,\text{in}} = \frac{P_{0,\text{in}}}{RT_{0,\text{in}}} \quad (29)$$

$$p_{\text{in}} = P_{0,\text{in}} (\tau_{\text{in}})^{-\frac{\gamma}{\gamma-1}} \quad (30)$$

Questa formulazione chiusa, accoppiata al calcolo della "faccia" zero nel solutore di Roe, rende l'ingresso immensamente robusto alle fluttuazioni di avvio rispetto alla classiche estrapolazioni lineari.

6.2 BC di Uscita (Outlet)

La BC all'uscita è ancorata al Regime Sonico del flusso all'ultima cella del dominio u_{N_x-1} rispetto a a_{N_x-1} . 1. ****Se Subsonico ($u < a$):**** L'informazione di pressione dall'ambiente esterno riesce a risalire il flusso e penetrare nel tubo. Pertanto, la pressione della ghost cell di destra viene bloccata artificialmente alla Pressione Ambiente: $p_{\text{out}} = P_{\text{amb}}$. Densità e velocità vengono espanse dalla cella adiacente interna (Gradiente nullo: $u_{\text{out}} = u_{N_x-1}$). 2. ****Se Supersonico ($u \geq a$):**** Nessuna informazione dall'esterno può propagarsi in controcorrente. La Pressione Ambiente è disconnessa fluidodinamicamente dal tubo. In questo caso, impostiamo $p_{\text{out}} = p_{N_x-1}$. Il solutore innesca così una condizione di "Fully Upwind Extrapolation", ed il gas esce imperturbato indipendentemente dalla pressione di scarico imposta.

7 Ottimizzazione Architetture Avanzata

Risolvere queste equazioni complesse spazialmente (per $N_x = 1000$ celle) e temporalmente per 100.000 loop richiede ingegneria algoritmica specifica per non superare decine di minuti di esecuzione in Python nativo.

7.1 JIT Compilation (Numba)

Il calcolo massivo, racchiuso nelle funzioni `roe_flux_numba_fixed` e `cf_d_core_loop`, è delegato a Numba. L'annotatore `@jit` "congela" il codice Python alla prima esecuzione, esaminandone i tipi primitivi (`float64`, array `numpy`) e compilandolo in basso livello tramite la macchina virtuale LLVM, trasformando il loop di `cf_d_core_loop` in esecuzione codice C/Assembly puro con velocità sbalorditive. L'opzione `fastmath=True` rilassa i rigorosi standard IEEE per i float floating-point (es. ignorando la stretta accuratezza nei NaN), consentendo l'uso massiccio delle istruzioni vettorializzate AVX dei processori Intel/AMD.

7.2 Gestione Memoria e Preallocazione

Allocare array numpy all'interno del "time loop" (creare `np.zeros(nx)` 100.000 volte in sequenza) porterebbe a un devastante carico del "Garbage Collector" e al blocco della compilazione JIT per assenza di heap predefinito. L'architettura del `general_solver.py` alloca **staticamente in testa** all'algoritmo (linee 59-63):

```
1 rho = np.zeros(nx)
2 u = np.zeros(nx)
3 p = np.zeros(nx)
4 a = np.zeros(nx)
5 F_int = np.zeros((3, nx + 1))
```

Dentro il ciclo temporale, le matrici non vengono mai ricreate, ma sempre aggiornate sul posto usando cicli per indicizzazione diretta: `rho[i] = ...`.

7.3 Multithreading Parallelo (prange)

Grazie al costrutto ai volumi finiti Esplicito, il calcolo della cella i ad un istante temporale dato *non* dipende dall'aggiornamento simultaneo delle altre celle (ma solo dai loro valori passati, che restano congelati in `U_curr`). Questo consente una totale indipendenza computazionale olistica per la porzione spaziale. Sostituendo il normale loop Python con `for i in prange(nx):` e includendo `parallel=True` nel decoratore, Numba inietta codice OpenMP per suddividere l'array da 1000 elementi in chunks. Se il PC ha 8 core, ognuno dei core calcolerà un chunk di 125 celle completamente in isolamento, accorpendo l'aggiornamento a fine passo temporale. Le prestazioni vengono ridotte in base al thread pool ad un fattore prossimo al numero di core CPU.

8 Mappatura Spaziale dei Componenti Eterogenei

Prima che il solver inizi, gli N componenti astratti del gasdotto (valvole, ugelli, tubazioni dritte) passati in formato UI JSON devono essere geometricamente schiacciati nella griglia monodimensionale x creata dinamicamente in `GeneralSolver1D.solve`.

L'algoritmo avanza mantenendo traccia di una coordinata corrente `curr_x`. Si cicla su ciascun componente valutandone la lunghezza L . Per mappare il componente sui Nodi (N_x) o Interfacce ($N_x + 1$), si usano maschere di posizionamento:

```
1 eps = dx * 1e-3
2 mask_c = (x >= curr_x - eps) & (x <= curr_x + L + eps)
3 mask_i = (x_int >= curr_x - eps) & (x_int <= curr_x + L + eps)
```

L'integrazione di un parametro "Epsilon" (ϵ) infinitesimale previene i bug catastrofici del calcolo galleggiante (floating point limits), in cui la frazione ≈ 0.9999999 non supererebbe ≥ 1.0 e la cella resterebbe non mappata ($\text{Area} = 0$, che fa deflagrare la divisione di massa).

Viene specificamente saltata l'imposizione geometrica della discontinuità "`normal_shock`" poiché la riproduzione fisica di un salto d'urto statico avverrà in automatico integrando le differenze finite nei passaggi CFD.

Concluse le iterazioni, l'array bidimensionale `U_final` viene ridistribuito nel formato `Mach`, `Temperature`, `Pressure` richiesto dal frontend Web, generando dizionari serializzabili pronti al tracciamento su Grafana o React Recharts.